

Операционные системы

Лекция 14

Кэширование данных

План лекции

- Кэширование данных
- Принцип действия кэш-памяти
- Отображение основной памяти на кэш
- Схемы выполнения запросов в системах с кэш-памятью
- Буфер ассоциативной трансляции

Кэширование данных

Кэширование данных – это универсальный метод ускорения доступа к данным, основанный на комбинации двух типов памяти, отличающихся временем доступа, объемом и стоимостью хранения данных

Наиболее часто используемая в данный период информация динамически копируется из «медленной, но большой» памяти в «быструю, но маленькую» **кэш-память**, или **кэш** (cash)

Кэширование – это процесс переноса данных в память с более быстрым доступом

Кэширование данных

Например, если кэширование применяется для уменьшения среднего времени доступа к оперативной памяти, то в качестве кэша выступает быстродействующая статическая память процессора или даже его регистры

А если система ввода-вывода кэширует данные для ускорения доступа к информации, хранящейся на диске, то в этом случае роль кэша исполняют буферы в оперативной памяти, в которых оседают наиболее активно используемые данные

Кэширование широко используется и в Интернете, в частности при разрешении символьных DNS-имен, в протоколе ARP, в работе браузера. Страницы информации с удаленных веб-сайтов («медленная, но большая» память), к которым с помощью браузера обращается пользователь, кэшируются на диске его компьютера («быстрая, но маленькая» память)

Кэширование данных

Кэширование позволяет не только сократить среднее время доступа к данным, но и экономить более дорогую быстродействующую память. Виртуальную память можно считать одним из вариантов реализации принципа кэширования данных, при котором основной целью является экономия памяти

Действительно, оперативная память выступает здесь в роли кэша по отношению к внешней памяти – жесткому диску. Кэширование задеиствуется здесь не для того, чтобы сократить время доступа к данным (команды в принципе не могут выполняться, находясь на диске), а для того, чтобы заставить диск частично подменить оперативную память за счет перемещения временно неиспользуемых кода и данных на диск с целью освобождения места для активных процессов

В результате наиболее интенсивно применяемые данные «оседают» в оперативной памяти, остальная же информация хранится в более объемной и менее дорогой стоящей внешней памяти

Кэширование данных

Неотъемлемым свойством кэш-памяти является её **прозрачность** для программ и пользователей

Система, реализующая кэширование, не требует никакой внешней информации об интенсивности обращения к данным; ни пользователи, ни программы не принимают никакого участия в перемещении данных из одного типа памяти в память другого типа

Все это делается автоматически системными средствами

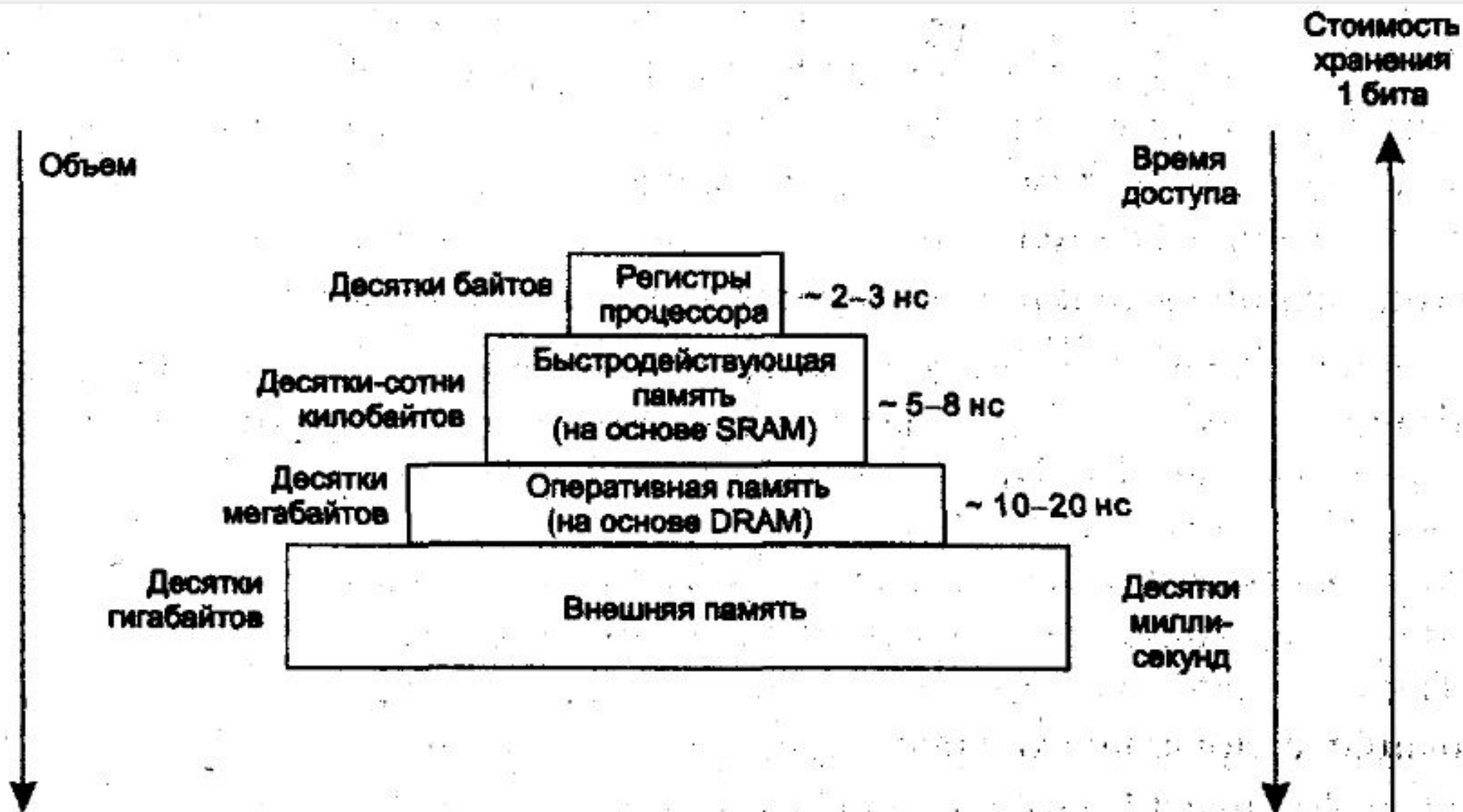
Кэширование данных

Память вычислительной машины представляет собой иерархию запоминающих устройств (ЗУ), отличающихся средним временем доступа к данным, объемом и стоимостью хранения одного бита

Фундаментом этой пирамиды запоминающих устройств служит внешняя память, как правило, представляемая жестким диском

Она имеет большой объем (десятки и сотни гигабайтов), но скорость доступа к данным является невысокой. Время доступа к диску измеряется миллисекундами

Кэширование данных



Кэширование данных

На следующем уровне располагается более быстродействующая (время доступа равно примерно 10-20 наносекундам) и менее объемная (от десятков мегабайтов до нескольких гигабайтов) оперативная память, реализуемая на относительно медленной динамической памяти DRAM

Для хранения данных, к которым необходимо обеспечить быстрый доступ, используются компактные быстродействующие запоминающие устройства на основе статической памяти SRAM, объем которых составляет от нескольких десятков до нескольких сотен килобайтов, а время доступа к данным обычно не превышает 8 нс

Кэширование данных

И наконец, верхушку в этой пирамиде составляют внутренние регистры процессора, которые также могут быть использованы для промежуточного хранения данных

Общий объем регистров составляет несколько десятков байтов, а время доступа определяется быстродействием процессора и равно в настоящее время примерно 2-3 нс

Все перечисленные характеристики ЗУ быстро изменяются по мере совершенствования вычислительной аппаратуры

В данном случае **важны не абсолютные значения** времени доступа или объема памяти, **а их соотношение** для разных типов запоминающих устройств

Кэширование данных

Таким образом, можно констатировать печальную закономерность – чем больше объем устройства, тем менее быстродействующим оно является

Более того, стоимость хранения данных в расчете на один бит также увеличивается с ростом быстродействия устройств

В то же время пользователю хотелось бы иметь и недорогую, и быструю память. Кэш-память представляет некоторое компромиссное решение этой проблемы

Принцип действия кэш-памяти

Рассмотрим одну из возможных схем кэширования
Содержимое кэш-памяти представляет собой совокупность **записей** обо всех загруженных в нее элементах данных из основной памяти

Каждая запись об элементе данных включает в себя:

- Значение элемента данных
- Адрес, который этот элемент данных имеет в основной памяти
- Дополнительную информацию, которая используется для реализации алгоритма замещения данных в кэше и обычно включает признак модификации и признак действительности данных

Принцип действия кэш-памяти



Структура кэш-памяти

Адрес данных в основной памяти	Данные	Управляющая информация

Принцип действия кэш-памяти

При каждом обращении к основной памяти по физическому адресу просматривается содержимое кэш-памяти с целью определения, не находятся ли там нужные данные

Кэш-память не является адресуемой, поэтому поиск нужных данных осуществляется по содержимому – по взятому из запроса значению поля адреса в оперативной памяти

Далее возможен один из двух вариантов развития событий:

- Если данные обнаруживаются в кэш-памяти, то есть происходит **кэш-попадание (cache-hit)**, они считываются из нее, и результат передается источнику запроса
- Если нужные данные отсутствуют в кэш-памяти, то есть происходит **кэш-промах (cache-miss)**, они считываются из основной памяти, передаются источнику запроса и одновременно с этим копируются в кэш-память

Принцип действия кэш-памяти

Интуитивно понятно, что эффективность кэширования зависит от вероятности кэш-попадания

Покажем это путем нахождения зависимости среднего времени доступа к основной памяти от вероятности кэш-попаданий. Пусть имеется основное запоминающее устройство со средним временем доступа к данным t_1 и кэш-память, имеющая время доступа t_2 , очевидно, что $t_2 < t_1$. Пусть t – среднее время доступа к данным в системе с кэш-памятью, а p – вероятность кэш-попадания

По формуле полной вероятности имеем:

$$t = t_1(1 - p) + t_2 p = (t_2 - t_1)p + t_1$$

Принцип действия кэш-памяти

Среднее время доступа к данным в системе с кэш-памятью линейно зависит от вероятности кэш-попадания и изменяется от среднего времени доступа в основное запоминающее устройство t_1 при $p = 0$ до среднего времени доступа непосредственно в кэш-память t_2 при $p = 1$

Отсюда видно, что кэширование имеет смысл только при высокой вероятности кэш-попадания

Вероятность обнаружения данных в кэше зависит от разных факторов, таких, например, как *объем кэша, объем кэшируемой памяти, алгоритм замещения данных в кэше, особенности выполняемой программы, время ее работы, уровень мультипрограммирования и других особенностей вычислительного процесса*

Принцип действия кэш-памяти

Тем не менее в большинстве реализаций кэш-памяти процент кэш-попаданий оказывается весьма высоким – свыше 90%

Такое высокое значение вероятности нахождения данных в кэш-памяти объясняется наличием у данных объективных свойств: **пространственной и временной локальности**

Временная локальность. Если произошло обращение по некоторому адресу, то следующее обращение по тому же адресу с большой вероятностью произойдет в ближайшее время

Пространственная локальность. Если произошло обращение по некоторому адресу, то с большой вероятностью в ближайшее время произойдет обращение к соседним адресам

Принцип действия кэш-памяти

Именно основываясь на свойстве временной локальности, данные, только что считанные из основной памяти, размещают в запоминающем устройстве быстрого доступа, предполагая, что скоро они опять понадобятся

Вначале работы системы, когда кэш-память еще пуста, почти каждый запрос к основной памяти выполняется «по полной программе»: просмотр кэша, констатация кэш-промаха, чтение данных из основной памяти, передача результата источнику запроса и копирование данных в кэш

Затем, по мере заполнения кэша, в полном соответствии со свойством временной локальности возрастает вероятность обращения к данным, которые уже были использованы на предыдущем этапе работы системы, то есть к данным, которые содержатся в кэше и могут быть считаны значительно быстрее, чем из основной памяти

Принцип действия кэш-памяти

Свойство пространственной локальности также используется для повышения вероятности кэш-попадания: как правило, в кэш-память считывается не один информационный элемент, к которому произошло обращение, а целый блок данных, расположенных в основной памяти в непосредственной близости с данным элементом

Поскольку при выполнении программы очень высока вероятность, что команды выбираются из памяти последовательно одна за другой из соседних ячеек, то имеет смысл загружать в кэш-память целый фрагмент программы

Аналогично, если программа ведет обработку некоторого массива данных, то ее работу можно ускорить, загрузив в кэш часть или даже весь массив данных. При этом учитывается высокая вероятность того, что значительное число обращений к памяти будет выполняться к адресам массива данных

Проблема согласования данных

В процессе работы содержимое кэш-памяти постоянно обновляется, а значит, время от времени данные из нее должны вытесняться

Вытеснение означает либо простое объявление свободной соответствующей области кэш-памяти (сброс бита действительности), если вытесняемые данные за время нахождения в кэше не были изменены, либо в дополнение к этому копирование данных в основную память, если они были модифицированы

Алгоритм замены данных в кэш-памяти существенно влияет на ее эффективность. В идеале такой алгоритм должен, во-первых, быть максимально быстрым, чтобы не замедлять работу кэш-памяти, во-вторых, обеспечивать максимально возможную вероятность кэш-попаданий

Проблема согласования данных

Поскольку из-за непредсказуемости вычислительного процесса ни один алгоритм замещения данных в кэш-памяти не может гарантировать *оптимальный* результат, разработчики ограничиваются *рациональными решениями*, которые, по крайней мере, не слишком замедляют работу кэша – запоминающего устройства, изначально призванного быть быстрым

Наличие в компьютере двух копий данных – в основной памяти и в кэше – порождает **проблему согласования данных**. Если происходит запись в основную память по некоторому адресу, а содержимое этой ячейки находится в кэше, то в результате соответствующая запись в кэше становится недостоверной

Проблема согласования данных

Рассмотрим два подхода к решению этой проблемы:

Сквозная запись (write through): при каждом запросе к основной памяти, в том числе и при записи, просматривается кэш

Если данные по запрашиваемому адресу отсутствуют, то запись выполняется только в основную память

Если же данные, к которым выполняется обращение, находятся в кэше, то запись выполняется одновременно в кэш и основную память

Проблема согласования данных

Обратная запись (write back): аналогично, при возникновении запроса к памяти выполняется просмотр кэша, и если запрашиваемых данных там нет, то запись выполняется только в основную память

В противном же случае запись производится только в кэш-память, при этом в описателе данных делается специальная отметка (признак модификации), которая указывает на то, что при вытеснении этих данных из кэша необходимо переписать их в основную память, чтобы актуализировать устаревшее содержимое основной памяти

Проблема согласования данных

В некоторых алгоритмах замещения предусматривается первоочередная выгрузка модифицированных или, как еще говорят, «грязных» данных

Модифицированные данные могут выгружаться не только при освобождении места в кэш-памяти для новых данных, но и в «фоновом режиме», когда система не очень загружена

Отображение основной памяти на кэш

Алгоритм поиска и алгоритм замещения данных в кэше непосредственно зависят от того, каким образом основная память отображается на кэш-память

Принцип прозрачности требует, чтобы правило отображения основной памяти на кэш память не зависело от работы программ и пользователей

При кэшировании данных из оперативной памяти широко используются две основных схемы отображения: **случайное** и **детерминированное**

Отображение основной памяти на кэш

При **случайном отображении** элемент оперативной памяти в общем случае может быть размещен в произвольном месте кэш-памяти

Для того чтобы в дальнейшем можно было найти нужные данные в кэше, они помещаются туда вместе со своим адресом, то есть тем адресом, который данные имеют в оперативной памяти

При каждом запросе к оперативной памяти выполняется поиск в кэше, причем критерием поиска выступает адрес оперативной памяти из запроса

Очевидная схема простого перебора для поиска нужных данных в случае кэша оказывается непригодной из-за недопустимо больших временных затрат

Отображение основной памяти на кэш

Для кэшей со случайным отображением используется так называемый **ассоциативный поиск**, при котором сравнение выполняется не последовательно с каждой записью кэша, а параллельно со всеми его записями

Признак, по которому выполняется сравнение, называется **тегом** (tag). В данном случае тегом является адрес данных в оперативной памяти

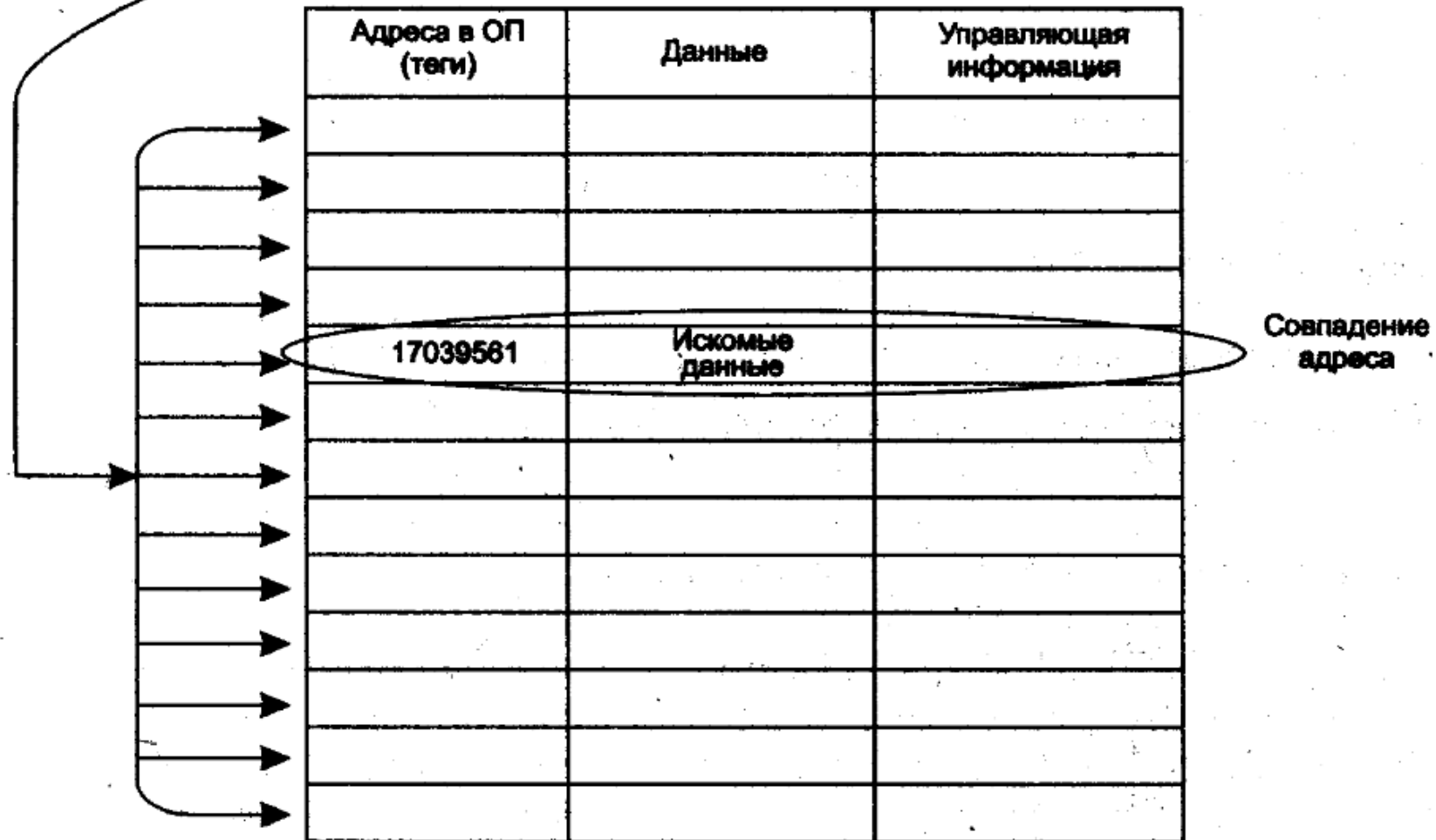
Электронная реализация такой схемы приводит к удорожанию памяти, причем стоимость существенно возрастает с увеличением объема запоминающего устройства. Поэтому ассоциативная кэш-память используется в тех случаях, когда для обеспечения высокого процента кэш-попаданий достаточно небольшого объема памяти

Адрес в ОП из запроса

17039561

Тег

Кэш-память



Параллельное
сравнение

Отображение основной памяти на кэш

В кэшах, построенных на основе случайного отображения, вытеснение старых данных происходит только в том случае, когда вся кэш-память заполнена и нет свободного места

Выбор данных на выгрузку осуществляется среди всех записей кэша. Обычно этот выбор основывается на тех же приемах, что и в алгоритмах замещения страниц, например, выгрузка данных, к которым дольше всего не было обращений, или данных, к которым было меньше всего обращений

Отображение основной памяти на кэш

Второй, **детерминированный**, способ отображения предполагает, что любой элемент основной памяти всегда отображается на одно и то же место в кэш-памяти

В этом случае кэш-память разделена на строки, каждая из которых предназначена для хранения одной записи об одном элементе данных (в действительности запись в кэше обычно содержит несколько элементов данных) и имеет свой номер

Между номерами строк кэш-памяти и адресами оперативной памяти устанавливается соответствие «один ко многим»: одному номеру строки соответствует несколько (обычно достаточно много) адресов оперативной памяти

Отображение основной памяти на кэш

Описанный здесь способ отображения является частным случаем хэширования, а соответствующий алгоритм – хэш-функцией

Хэш-функции требуются для решения самых разных задач, например, они широко используются в криптографии

Приближенным аналогом хэширования является размещение слов в справочнике по алфавиту

Хэш-функцией здесь является отделение первой буквы, которая определяет раздел справочника. Далее можно вести поиск только в пределах этого раздела

Отображение основной памяти на кэш

В качестве отображающей функции может использоваться простое выделение нескольких разрядов из адреса оперативной памяти, которые интерпретируются как номер строки кэш-памяти (такое отображение называется **прямым**)

Например, пусть в кэш-памяти может храниться 1024 записи, то есть кэш имеет 1024 строки, пронумерованных от 0 до 1023

Тогда любой адрес оперативной памяти может быть отображен на адрес кэш-памяти путем использования лишь 10 младших двоичных разрядов

Адрес ОП из запроса

1110011101011

1101100011

Тег

Номер элемента
кэш-памяти

Кэш

Память тегов

Память данных

00...0

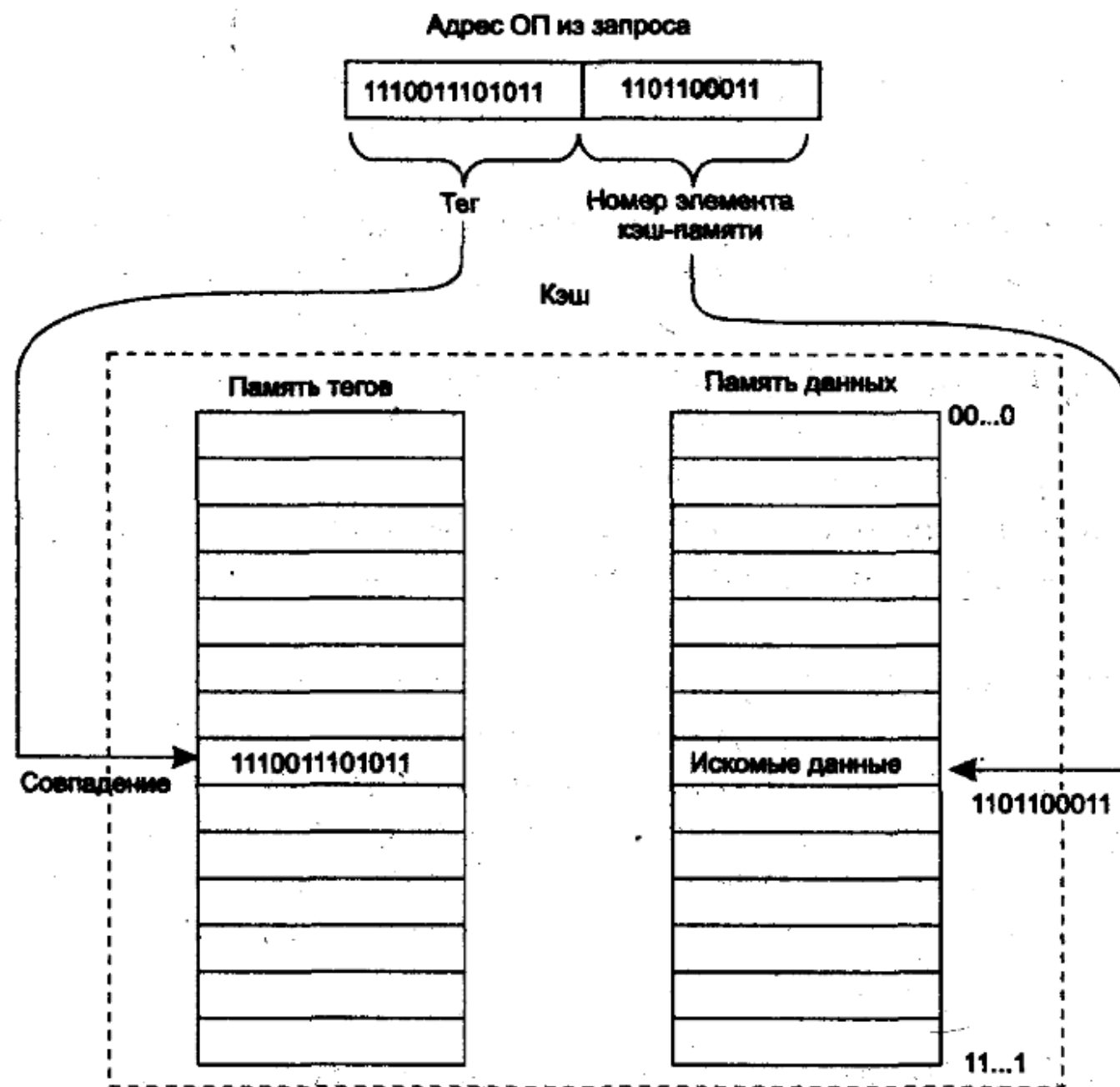
Совпадение

1110011101011

Искомые данные

1101100011

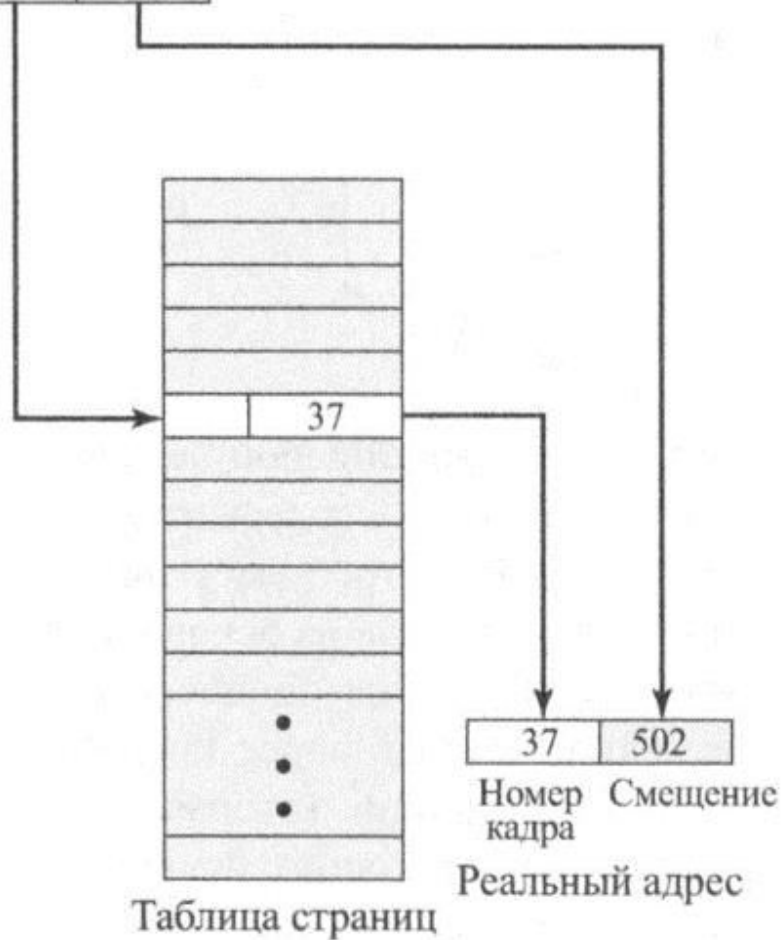
11...1



Виртуальный адрес

Номер
страницы Смещение

5	502
---	-----

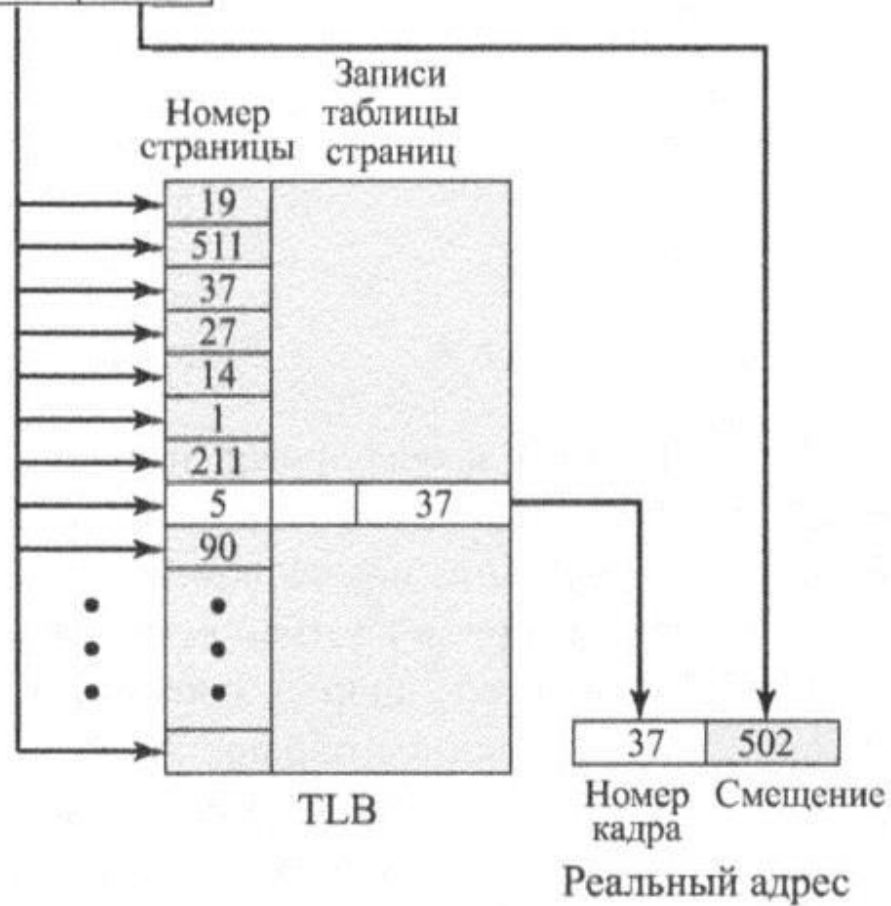


а) Прямое отображение

Виртуальный адрес

Номер
страницы Смещение

5	502
---	-----



б) Ассоциативное отображение

Отображение основной памяти на кэш

Хэширование дает возможность выполнить быстрый поиск данных в кэше при поступлении запроса. Адрес оперативной памяти, указанный в запросе, обрабатывается с использованием той же хэш-функции, которой в данном случае является отбрасывание всех старших двоичных разрядов

Младшие 10 разрядов интерпретируются как номер строки в кэш-памяти, и, таким образом, исчезает необходимость просматривать всю кэш-память

Отображение основной памяти на кэш

Однако поскольку в найденной строке могут находиться данные из любой ячейки оперативной памяти, младшие разряды адреса которой совпадают с номером строки, необходимо выполнить дополнительную проверку

Для этих целей каждая строка кэш-памяти дополняется тегом, содержащим старшую часть адреса данных в оперативной памяти. При совпадении тега с соответствующей частью адреса из запроса констатируется кэш-попадание

Отображение основной памяти на кэш

Если же происходит кэш-промах, данные считываются из оперативной памяти и копируются в кэш

Если строка кэш-памяти, в которую должен быть скопирован элемент данных из оперативной памяти, содержит, другие данные, то последние вытесняются из кэша

Заметим, что процесс замещения данных в кэш-памяти на основе прямого отображения существенно отличается от процесса замещения данных в кэш-памяти со случайным отображением

Во-первых, вытеснение данных происходит не только в случае отсутствия свободного места в кэше

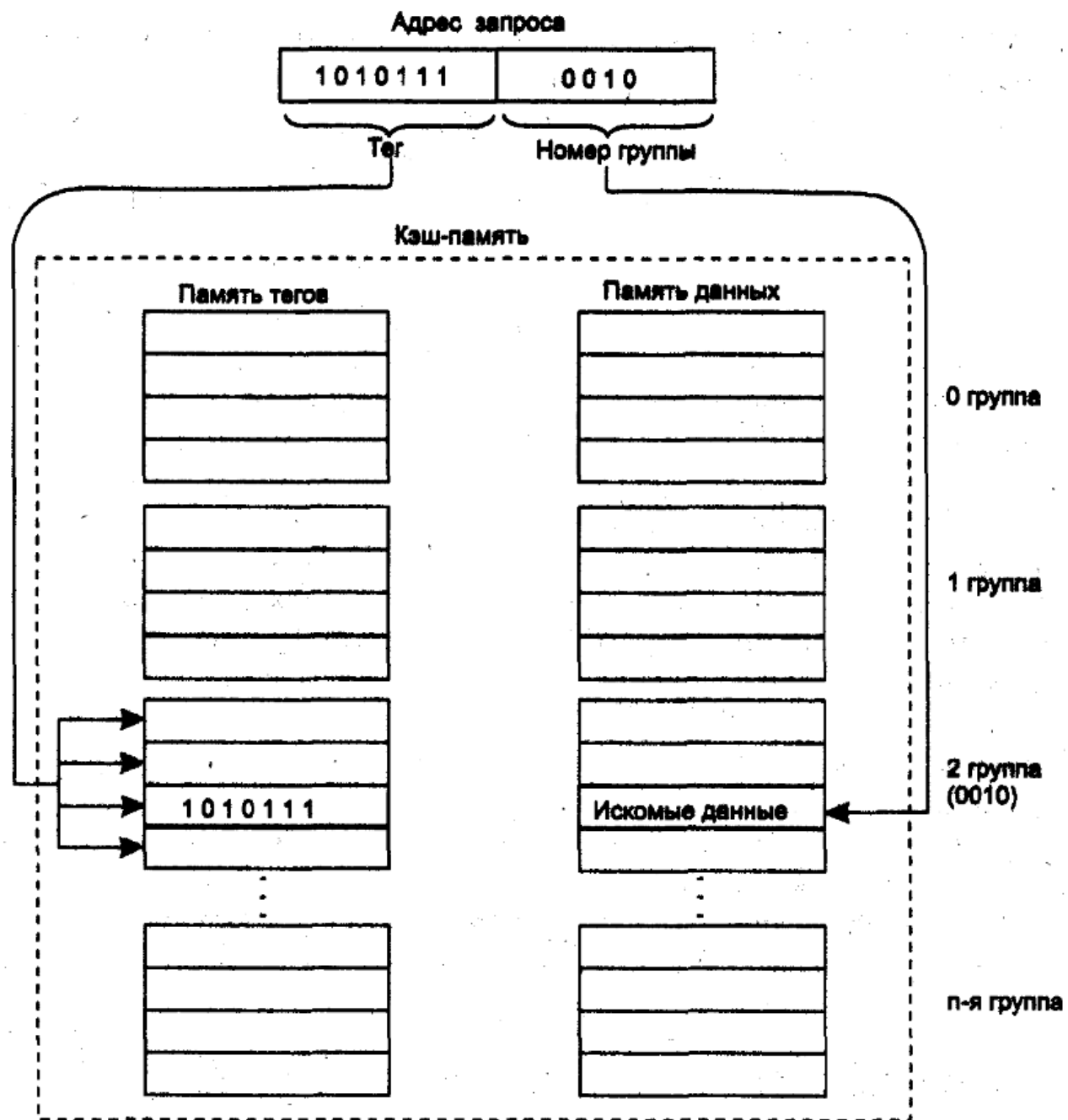
Во-вторых, никакого выбора данных для замещения не происходит

Отображение основной памяти на кэш

Во многих современных процессорах кэш-память строится на основе сочетания этих двух подходов, что позволяет найти компромисс между сравнительно низкой стоимостью кэша с прямым отображением и интеллектуальностью алгоритмов замещения в кэше со случайным отображением

При **смешанном подходе** произвольный адрес оперативной памяти отображается не на один адрес кэш-памяти (что характерно для прямого отображения) и не на любой адрес кэш-памяти (как это делается при случайном отображении), а на некоторую группу адресов. Все группы пронумерованы

Поиск в кэше осуществляется вначале по номеру группы, полученному из адреса оперативной памяти в запросе, а затем в пределах группы путем ассоциативного просмотра всех записей в группе на предмет совпадения старших частей адресов оперативной памяти



Отображение основной памяти на кэш

При промахе данные копируются по любому свободному адресу из однозначно заданной группы

Если свободных адресов в группе нет, то выполняется вытеснение данных. Поскольку кандидатов на выгрузку несколько (все записи данной группы), алгоритм замещения может учесть интенсивность обращений к данным и тем самым повысить вероятность попаданий в будущем

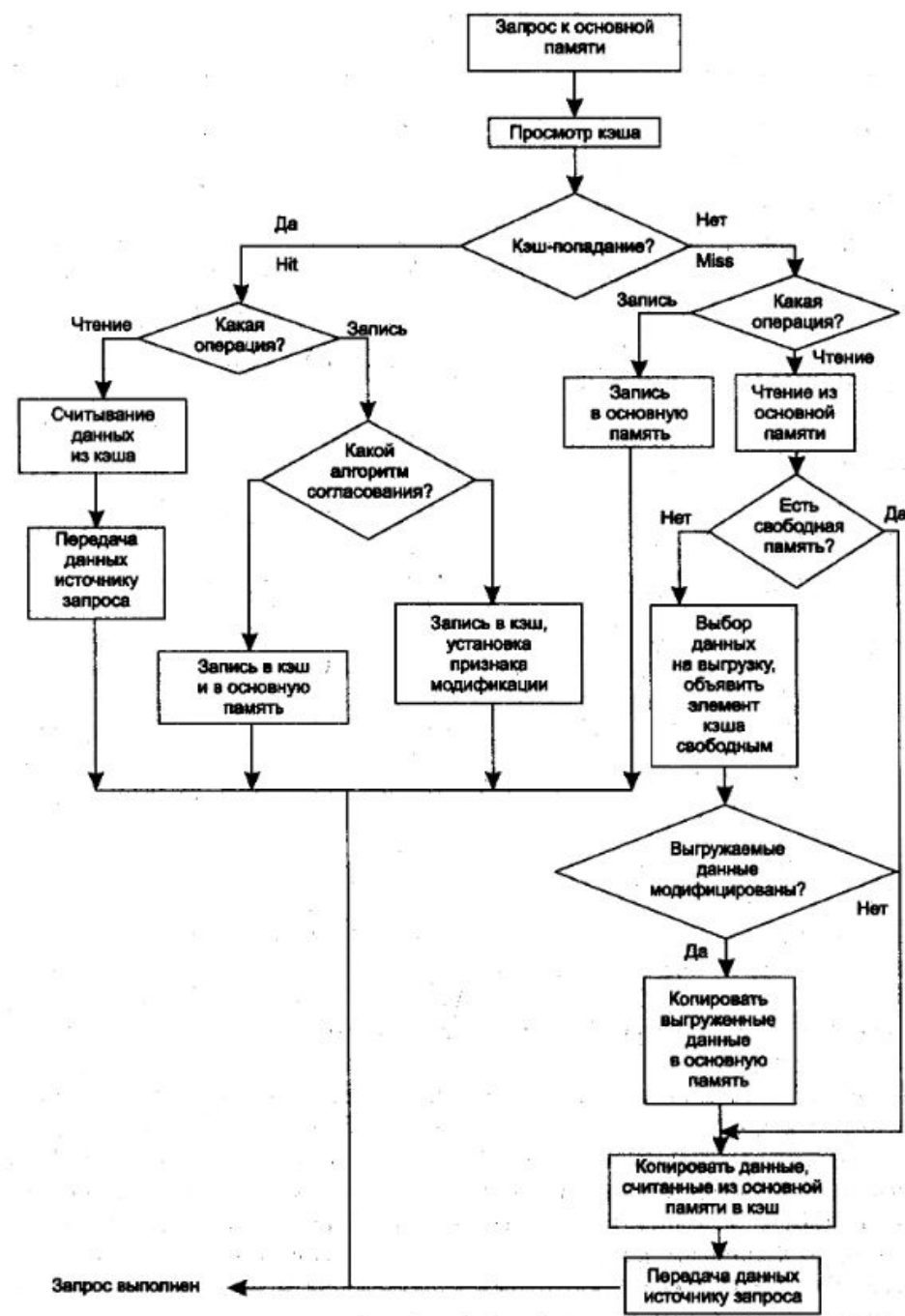
Таким образом, в данном способе комбинируется прямое отображение на группу и случайное отображение в пределах группы

Схемы выполнения запросов в системах с кэш-памятью

На следующей схеме приведена обобщенная схема работы кэш-памяти. Большая часть ветвей этой схемы уже была подробно рассмотрена ранее, поэтому остановимся только на некоторых особых случаях

Из схемы видно, что когда выполняется запись, кэш просматривается только с целью согласования содержимого кэша и основной памяти

Если происходит промах, то запросы на запись не вызывают никаких изменений содержимого кэша. В некоторых же реализациях кэш-памяти при отсутствии данных в кэше они копируются туда из основной памяти независимо от того, выполняется запрос на чтение или на запись



Схемы выполнения запросов в системах с кэш-памятью

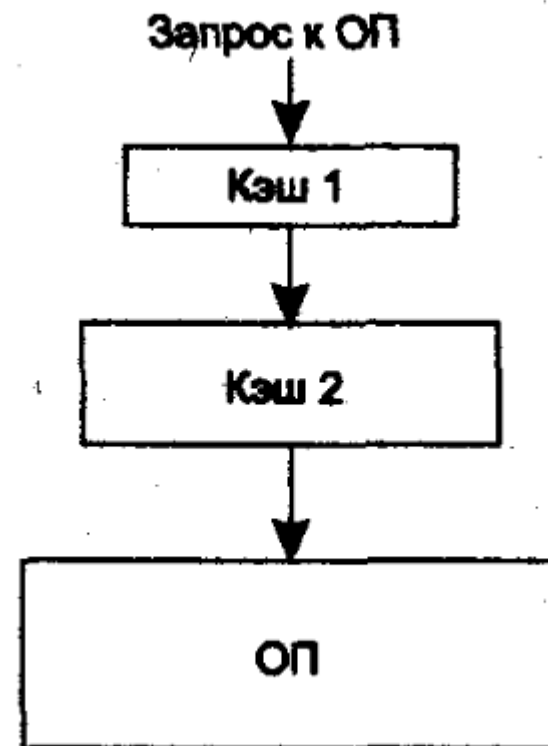
В соответствии с описанной логикой работы кэш-памяти следует, что при возникновении запроса сначала просматривается кэш, а затем, если имеет место промах, выполняется обращение к основной памяти

Однако часто реализуется и другая схема работы кэша: поиск в кэше и в основной памяти начинается одновременно, а затем, в зависимости от результата просмотра кэша, операция в основной памяти либо продолжается, либо прерывается

Схемы выполнения запросов в системах с кэш-памятью

При выполнении запросов к оперативной памяти во многих вычислительных системах происходит двухуровневое кэширование

Кэш первого уровня имеет меньший объем и более высокое быстродействие, чем кэш второго уровня. Кэш второго уровня играет роль основной памяти по отношению к кэшу первого уровня

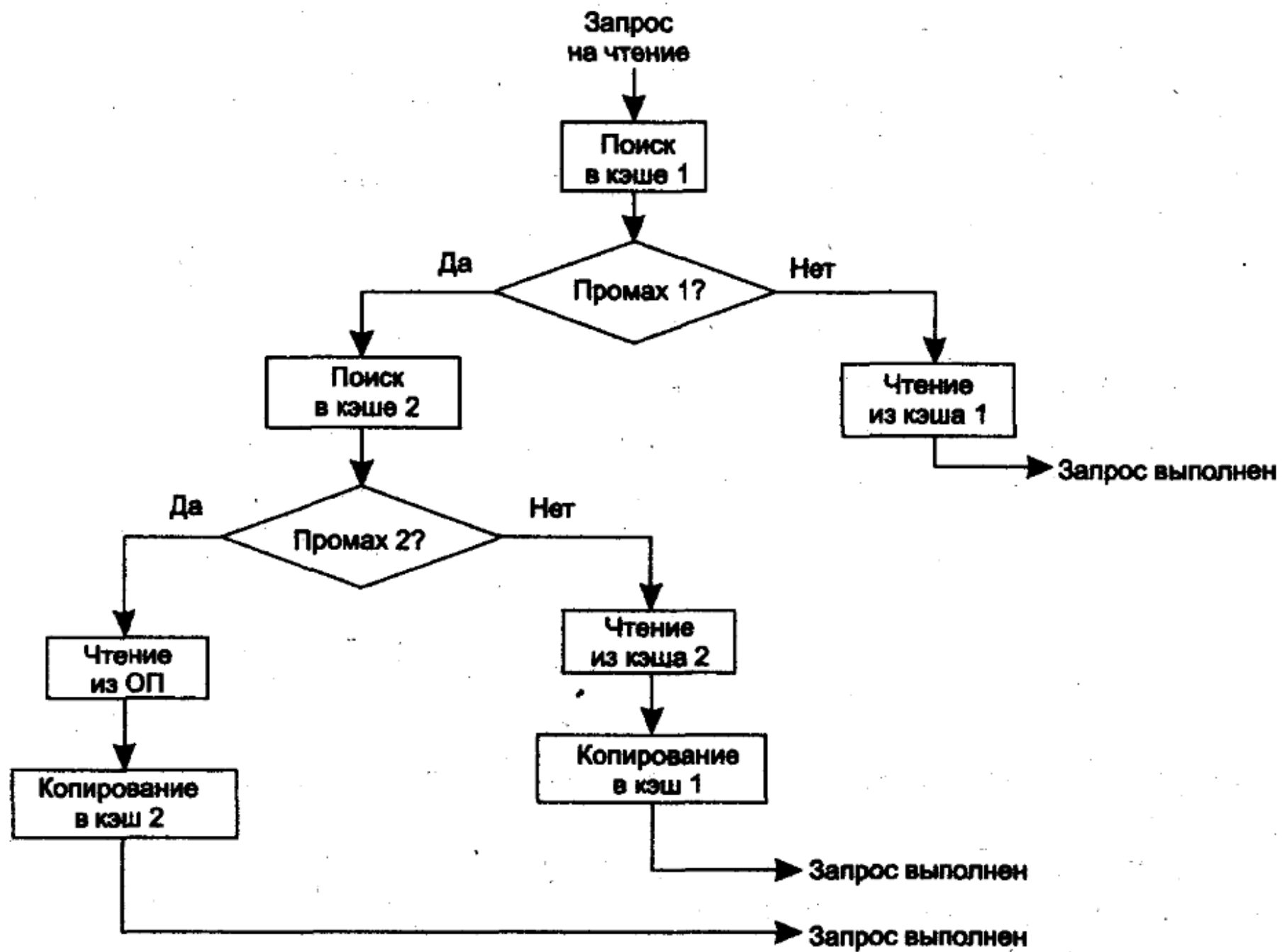


Схемы выполнения запросов в системах с кэш-памятью

На следующей схеме показана схема выполнения запроса на чтение в системе с двухуровневым кэшем

Сначала делается попытка обнаружить данные в кэше первого уровня. Если произошел промах, поиск продолжается в кэше второго уровня. Если же нужные данные отсутствуют и здесь, тогда происходит считывание данных из основной памяти

Понятно, что время доступа к данным оказывается минимальным, когда кэш-попадание происходит уже на первом уровне, несколько большим при обнаружении данных на втором уровне и обычным временем доступа к оперативной памяти, если нужных данных нет ни в том, ни в другом кэше

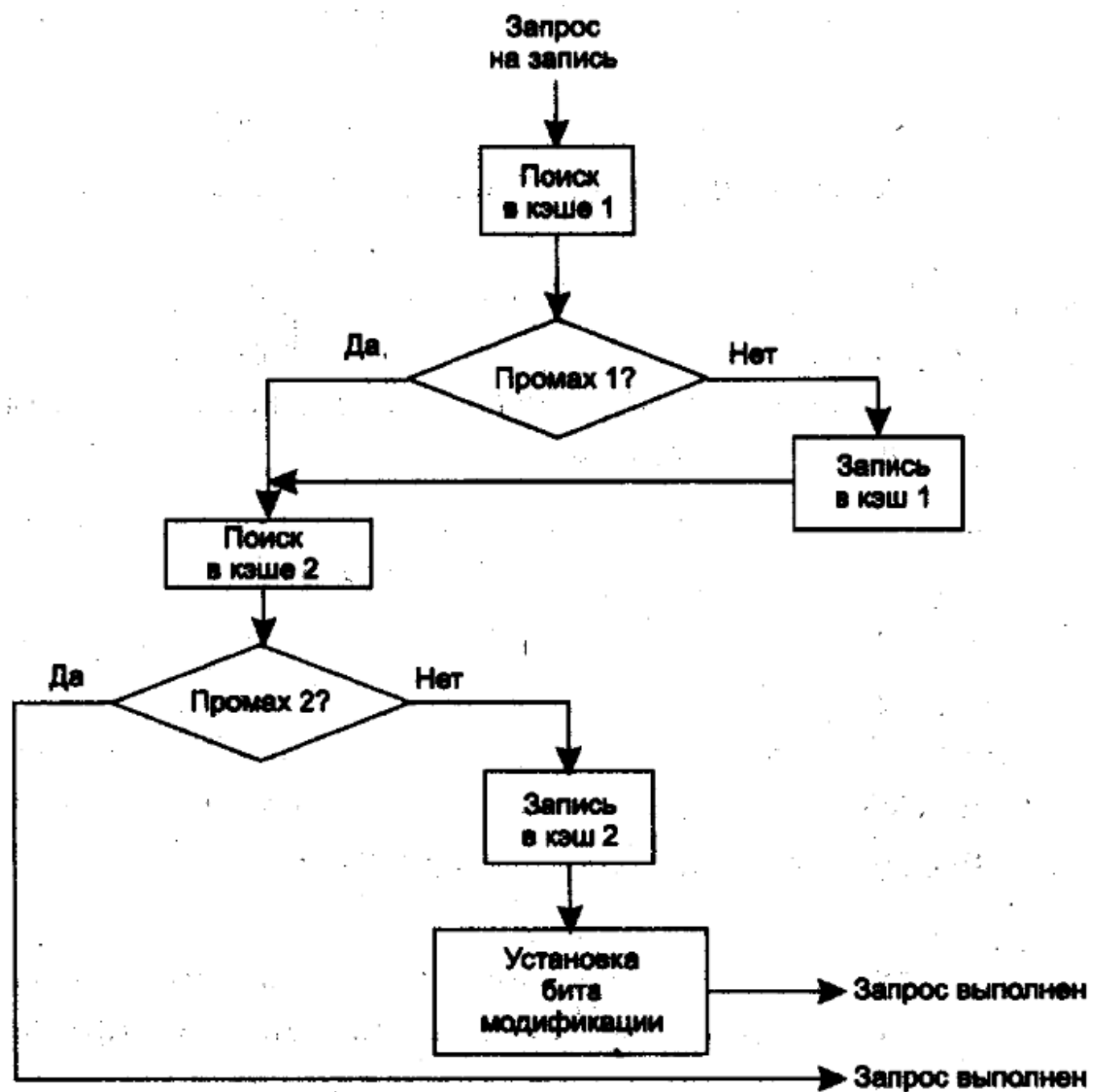


Схемы выполнения запросов в системах с кэш-памятью

При считывании данных из оперативной памяти происходит их копирование в кэш второго уровня, а если данные считываются из кэша второго уровня, то они копируются в кэш первого уровня

При работе такой иерархически организованной памяти необходимо обеспечить непротиворечивость данных на всех уровнях. Кэши разных уровней могут согласовывать данные разными способами. Пусть, например, кэш первого уровня использует сквозную запись, а кэш второго уровня – обратную запись

На следующей схеме приведена схема выполнения запроса на запись в такой системе. При модификации данных необходимо убедиться, что они отсутствуют в кэшах. В этом случае выполняется запись только в оперативную память



Схемы выполнения запросов в системах с кэш-памятью

Если данные обнаруживаются в кэше первого уровня, то вступает в силу алгоритм сквозной записи: выполняется запись в кэш первого уровня и передается запрос на запись в кэш второго уровня, играющему в данном случае роль основной памяти

Запись в кэш второго уровня в соответствии с алгоритмом обратной записи, принятом на данном уровне, сопровождается установкой признака модификации, при этом никакой записи в оперативную память не производится

Если данные находятся в кэше второго уровня, то, так же как и в предыдущем случае, выполняется запись в этот кэш и устанавливается признак модификации

Буфер ассоциативной трансляции

Кэширование пар номеров виртуальных и физических страниц в буфере ассоциативной трансляции (Translation Lookaside Buffer, TLB) позволяет ускорять преобразование виртуальных адресов в физические при сегментно-страничной организации памяти

TLB представляет собой ассоциативную память небольшого объема, предназначенную для хранения интенсивно используемых дескрипторов страниц

В современных системах имеются отдельные буферы TLB для инструкций и данных

Буфер ассоциативной трансляции

В буфере TLB кэшируются дескрипторы страниц из таблицы страниц

Для хранения дескриптора в кэше отводится одна строка. Каждая строка дополнена тегом, в котором содержится номер соответствующей виртуальной страницы

Строки объединены по четыре в группы, называемые **наборами**

Таблица TLB, используемая для преобразования адресов инструкций, имеет 32 строки и соответственно 8 наборов

Номер набора называют **индексом** (index)

Таким образом, путем кэширования может быть получен физический адрес для доступа к 32 страницам памяти, содержащим инструкции

31		15	14	13	12		0
----	--	----	----	----	----	--	---

Индекс

Кэш-память

бит действительности

Память тегов

Память дескрипторов

b_0	v_0	L0
b_1	v_1	L1
b_2	v_2	L2
	v_3	L3

b_0	v_0	L0	
b_1	v_1	L1	
b_2	v_2	L2	
	v_3	L3	

b_0	v_0	L0	
b_1	v_1	L1	
b_2	v_2	L2	
	v_3	L3	

Набор 7

Память дескрипторов

Буфер ассоциативной трансляции

После того как механизмом сегментации получен линейный адрес, он должен быть преобразован в физический адрес

Для этого, прежде всего, необходимо найти дескриптор страницы, которой принадлежит данный адрес, и извлечь из него номер физической страницы

Обычная процедура предусматривает обращение к таблице разделов, а затем – к таблице страниц. Однако физический адрес может быть получен гораздо быстрее благодаря тому, что в буфере TLB хранятся копии дескрипторов наиболее интенсивно используемых страниц

Поэтому перед тем, как начать сравнительно длительную процедуру преобразования адресов, делается попытка обнаружить нужный дескриптор страницы в быстрой ассоциативной памяти TLB. Затем на основании номера физической страницы, полученного из TLB, вычисляется физический адрес

Буфер ассоциативной трансляции

При поиске данных в TLB используется линейный виртуальный адрес. Разряды 12-14 используются как индекс набора

Далее проверяются биты действительности всех строк выбранного набора. В начале работы кэш-памяти биты действительности всех строк сбрасываются в нуль. В результате работы алгоритма замещения бит действительности принимает значение 1, когда в соответствующей строке содержится достоверная информация, и сбрасывается в нуль, когда строка объявляется свободной

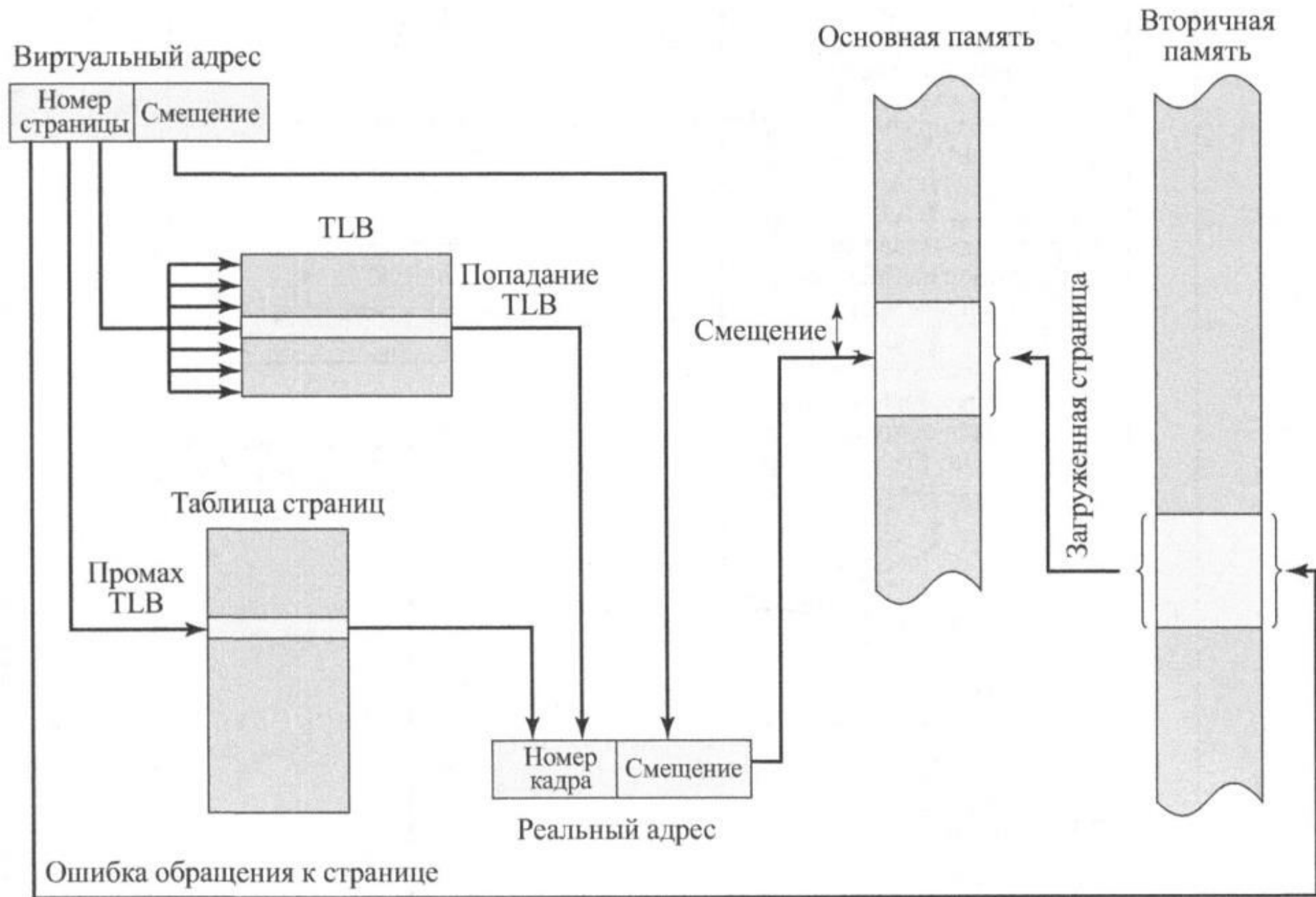
Для всех действительных строк выполняется ассоциативная процедура сравнения тегов со старшими разрядами (15-31 разряд) линейного виртуального адреса. Если произошло кэш-попадание, то номер физической страницы быстро поступает в схему формирования физического адреса

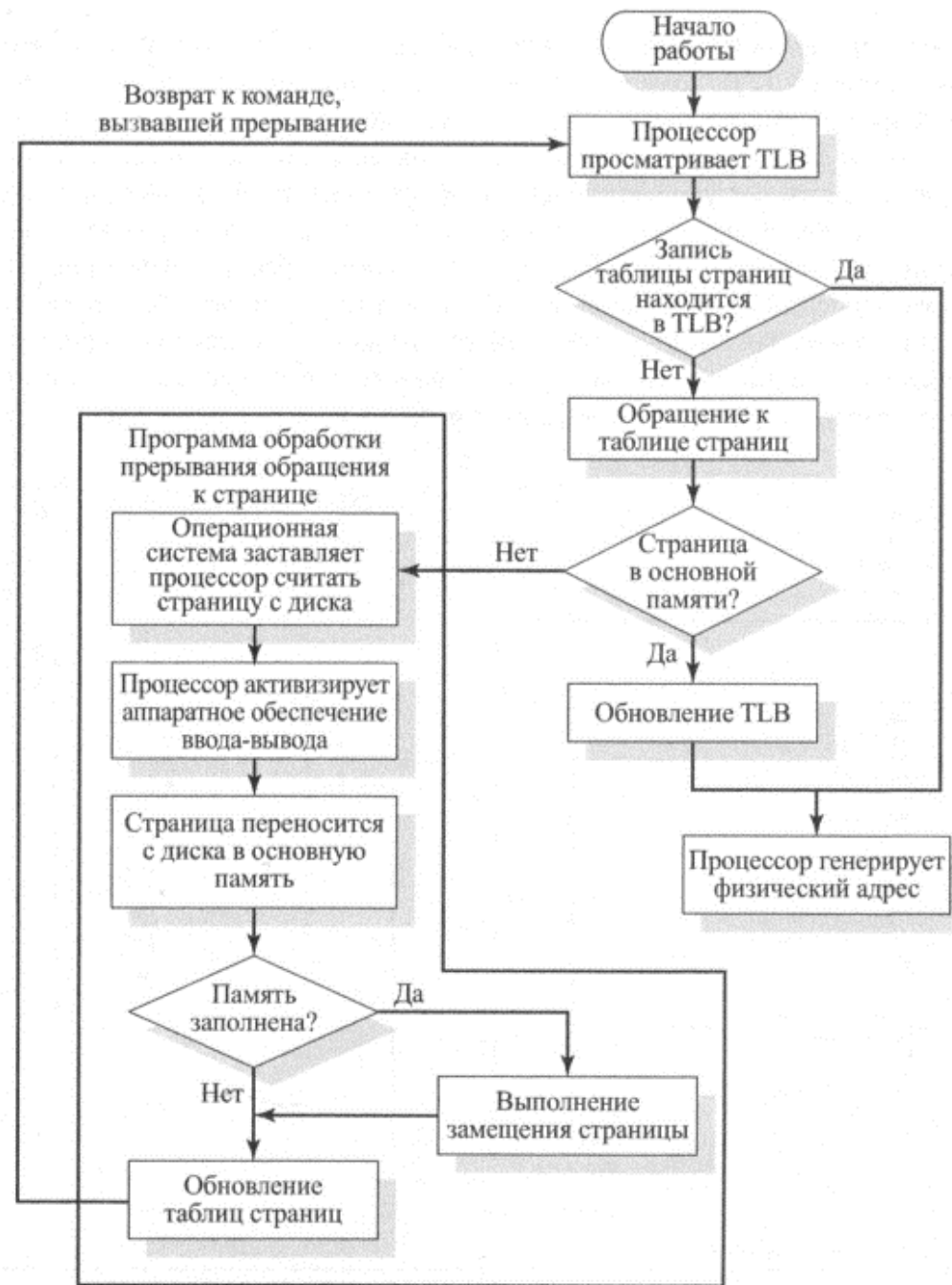
Буфер ассоциативной трансляции

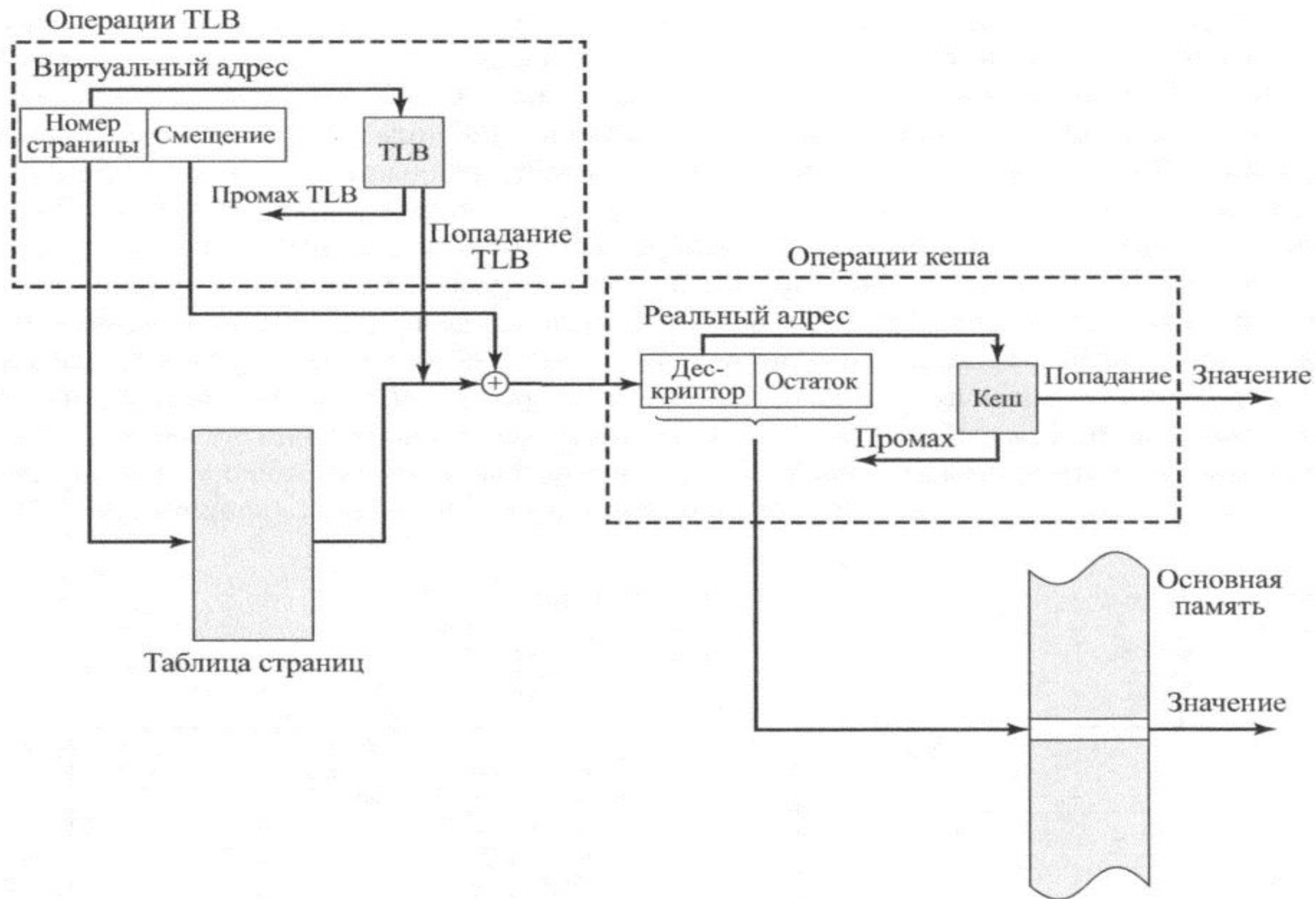
Если произошел промах и нужного дескриптора в TLB нет, то запускается многоэтапная процедура преобразования адреса, включающая обращения к таблицам разделов и страниц

Когда нужный дескриптор отыскивается в таблице страниц, он копируется в TLB. Номер набора, в который записывается кэшируемый дескриптор, определяется тремя младшими разрядами номера виртуальной страницы (разряды 12-14 линейного виртуального адреса)

Однако поскольку в наборе имеется четыре строки, необходимо определить, в какую именно надо поместить кэшируемые данные, дескриптор записывается либо в первую попавшуюся свободную строку, либо, если все строки заняты, в строку, к которой дольше всего не обращались







Буфер ассоциативной трансляции

Таким образом, в буфере TLB используется комбинированный способ отображения кэшируемых данных на кэш-память: прямое отображение дескрипторов на наборы и случайное отображение на строки в пределах набора

Наличие TLB позволяет в подавляющем числе случаев заменить сравнительно долгую процедуру преобразования адресов, связанную с несколькими обращениями к оперативной памяти, быстрым поиском в ассоциативной памяти

Операционные системы

Лекция 14

Кэширование данных